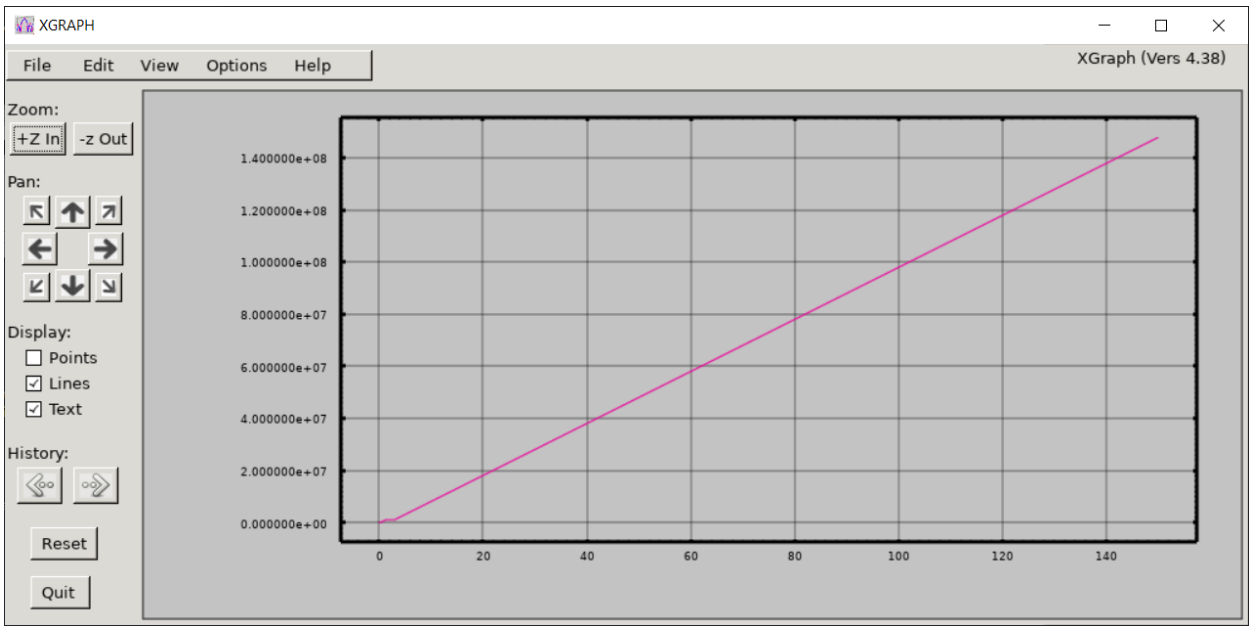
```
#include <sys/socket.h>
#include <arpa/inet.h>
#include <pthread.h>

#define timeoutValue 5
#define packetCount 10
#define windowSize 3

typedef struct Packet
{
    int id;
    int sent;
} Packet;

Packet packets[packetCount];
pthread_t thread[packetCount], receiveThread;
int sockfd;
struct sockaddr_in addr;
int windowStart, windowEnd;
int nextToSend;

void *sendPacket(void *sendPacket)
{
    Packet *packet = (Packet *)sendPacket;
    char buffer[1024];

    while (packet->sent == 0)
    {
        printf("Client: Sending packet %d\n",packet->id);
        bzero(buffer, 1024);
        sprintf(buffer, "%d", packet->id);

        sendto(sockfd, buffer, 1024, 0, (struct sockaddr *)&addr, sizeof(addr));
        sleep(timeoutValue);
        if (packet->sent == 0)
            printf("Client: Timeout for packet %d\n", packet->id);
    }
}

void *receivePacket()
{
    socklen_t addr_size = sizeof(addr);
    char buffer[1024];

    while (1)
    {
        bzero(buffer, 1024);
        recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr *)&addr, &addr_size);

        char msg[20];
        int packetID;

        sscanf(buffer, "%s%d", msg, &packetID);
        if (strcmp(msg, "NACK") == 0)
        {
            printf("Client: Received negative acknowledgment for packet %d\nSending again\n", packetID);
            pthread_cancel(thread[packetID - 1]);
            pthread_create(&thread[packetID - 1], NULL, sendPacket, (void *)&packets[packetID - 1]);
        }
        else if (strcmp(msg, "ACK") == 0)
        {
            printf("Client: Received acknowledgement for packet %d\n", packetID);
            packets[packetID - 1].sent = 1;
            if (windowStart == packetID - 1)
            {
                while (packets[windowStart].sent == 1)
                {
                    windowStart++;
                    if (windowEnd < packetCount)
                        windowEnd++;
                }
            }
        }
```


XGRAPH
File
Edit
View
Options
Help
XGraph (Vers 4.38)
Zoom:
+Z In
-z Out
Pan:
Display:
Points
Lines
Text
History:
Reset
Quit
1.400000e+08
1.200000e+08
1.000000e+08
8.000000e+07
6.000000e+07
4.000000e+07
2.000000e+07
0.000000e+00
0
20
40
60
80
100
120
140

```
            {
                remainingSize += packets[i];
                printf("\n\nIncoming Packet size: %d", packets[i]);
                printf("\nBytes remaining to Transmit: %d", remainingSize);
            }
            ++i;
        }

        timeToTransmit = (rand() % 4 + 1) * 10;
        printf("\nTime left for transmission: %d units\n", timeToTransmit);

        for (clk = 10; clk <= timeToTransmit; clk += 10)
        {
            sleep(1);
            if (remainingSize)
            {
                if (remainingSize <= rate)
                    op = remainingSize, remainingSize = 0;
                else
                    op = rate, remainingSize -= rate;

                printf("\nPacket %d transmitted\n", op);
                printf("Bytes Remaining to Transmit: %d\n", remainingSize);
            }
            else
            {
                printf("\nTime left for transmission: %d units", timeToTransmit - clk);
                printf("\nNo packets to transmit!!\n");
            }
        }
    }
}
```

Output

```
→  5.1 ./leaky

Enter the Output rate:5
Enter the Bucket Size:100


Incoming Packet size: 50
Bytes remaining to Transmit: 50
Time left for transmission: 30 units

Packet 5 transmitted
Bytes Remaining to Transmit: 45

Packet 5 transmitted
Bytes Remaining to Transmit: 40

Packet 5 transmitted
Bytes Remaining to Transmit: 35


Incoming Packet size: 30
Bytes remaining to Transmit: 65
Time left for transmission: 40 units

Packet 5 transmitted
Bytes Remaining to Transmit: 60
```

Result

The algorithm for Leaky Bucket protocol was implemented.

Simulate the transmission of ping messages over a star network topology consisting of 'n' nodes and find the number of packets dropped due to congestion using the NS2 simulator

Taking n = 6

Program

```
set ns [new Simulator]
set tf [open starTop.tr w]
$ns trace-all $tf

set nf [open starTop.nam w]
$ns namtrace-all $nf

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

$n0 label "Ping0"
$n4 label "Ping4"
$n5 label "Ping5"
$n6 label "Ping6"
$n2 label "Router"

$ns color 1 "red"
$ns color 2 "green"

$ns duplex-link $n0 $n2 100Mb 300ms DropTail
$ns duplex-link $n1 $n2 1Mb 300ms DropTail
$ns duplex-link $n3 $n2 1Mb 300ms DropTail

$ns duplex-link $n5 $n2 100Mb 300ms DropTail
$ns duplex-link $n2 $n4 1Mb 300ms DropTail
$ns duplex-link $n2 $n6 1Mb 300ms DropTail

$ns queue-limit $n0 $n2 5
$ns queue-limit $n2 $n4 3
$ns queue-limit $n2 $n6 2
$ns queue-limit $n5 $n2 5

set ping0 [new Agent/Ping]
$ns attach-agent $n0 $ping0

set ping4 [new Agent/Ping]
$ns attach-agent $n4 $ping4

set ping5 [new Agent/Ping]
$ns attach-agent $n5 $ping5

set ping6 [new Agent/Ping]
$ns attach-agent $n6 $ping6

$ping0 set packetSize_ 50000
$ping0 set interval_ 0.0001
```

```
enp0s25:
addresses:
- 192.168.0.100/24
gateway4: 192.168.0.1
nameservers:

search: [mydomain, otherdomain]
addresses: [1.1.1.1, 8.8.8.8, 4.4.4.4]
```

The search option can also be used with multiple domain names so that DNS queries will be appended in the order in which they are entered. For example, your network may have multiple sub-domains to search; a parent domain of example.com, and two sub-domains, sales.example.com and dev.example.com.

If you have multiple domains you wish to search, your configuration might look like the following:

```
network:
version: 2
renderer: networkd
ethernets:
enp0s25:
addresses:
- 192.168.0.100/24
gateway4: 192.168.0.1
nameservers:

search: [example.com, sales.example.com,

dev.example.com]

addresses: [1.1.1.1, 8.8.8.8, 4.4.4.4]
```

If you try to ping a host with the name of server1, your system will automatically query DNS for its Fully Qualified Domain Name (FQDN) in the following order:
1. server1.example.com
2. server1.sales.example.com
3. server1.dev.example.com

If no matches are found, the DNS server will provide a result of notfound and the DNS query will fail.

Static Hostnames

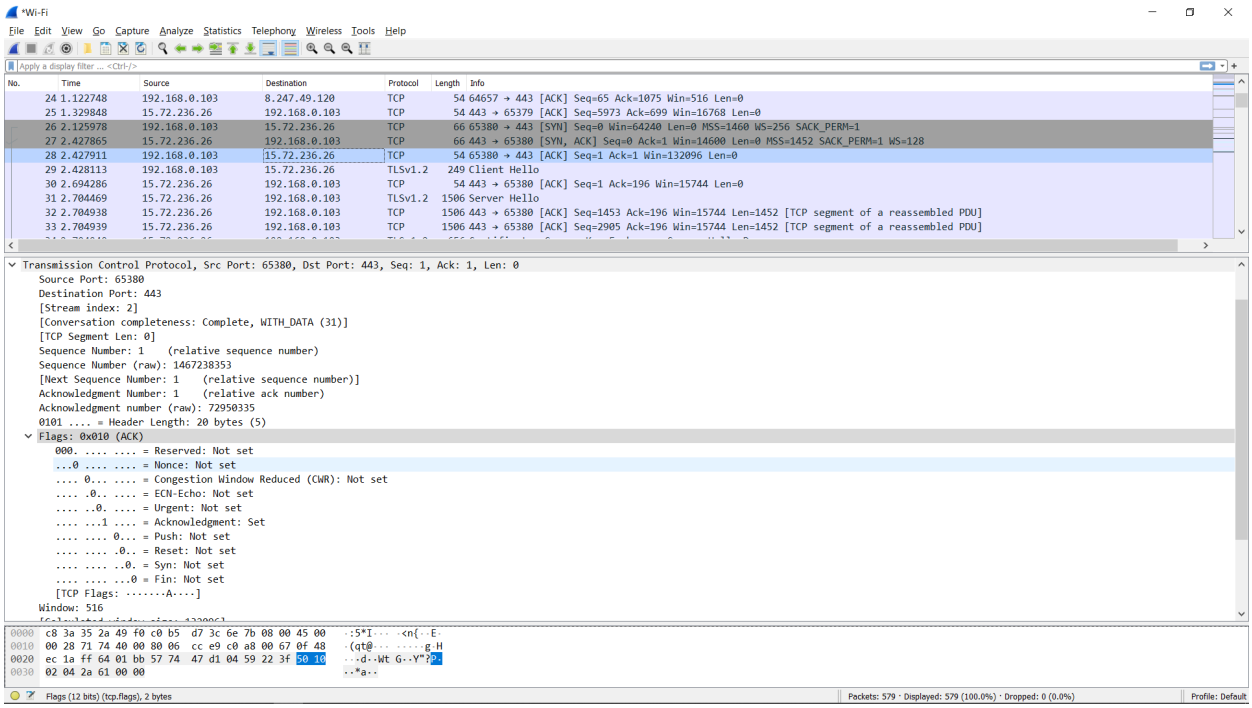
Static hostnames are locally defined hostname-to-IP mappings located in the file /etc/hosts. Entries in the hosts file will have precedence over DNS by default. This means that if your system tries to resolve a hostname and it matches an entry in /etc/hosts, it will not attempt to look up the record in DNS. In some configurations, especially when Internet access is not required, servers that communicate with a limited number of resources can be conveniently set to use static hostnames instead of DNS.

The following is an example of a hosts file where a number of local servers have been identified by simple hostnames, aliases and their equivalent Fully Qualified Domain Names (FQDN's).

127.0.0.1 localhost
127.0.1.1 ubuntu-server
10.0.0.11 server1 server1.example.com vpn
10.0.0.12 server2 server2.example.com mail
10.0.0.13 server3 server3.example.com www
10.0.0.14 server4 server4.example.com file

Name Service Switch Configuration

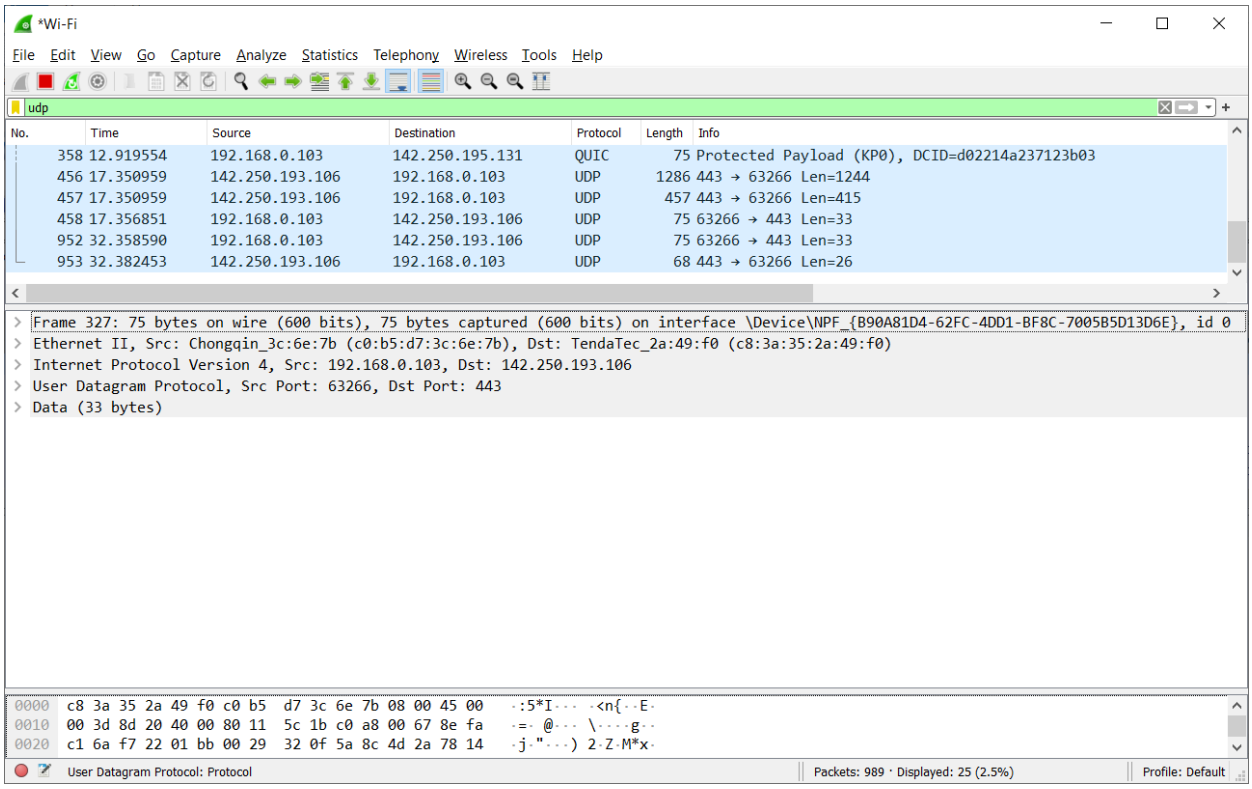
The order in which your system selects a method of resolving hostnames to IP addresses is controlled by the Name Service Switch (NSS) configuration file /etc/nsswitch.conf. As mentioned in the previous section,



Client acknowledges server's sequence number

Filters

Filters are used to filter out packets that satisfy a particular condition(s)



Filter to capture packets with UDP protocol

Experiment no: 5.1
Date: 22.07.22

Leaky Bucket

Aim

Implement Leaky Bucket Algorithm for congestion control.

Theory

Leaky bucket is a congestion control algorithm, used to maintain constant rate of data flow in a network. If we take a bucket with a hole at the bottom, and pour water into it, no matter at what rate water is poured into the bucket, it escapes through the hole at a constant rate. If the bucket is full, water poured into it leaks through the sides. This is the principle behind leaky bucket. It is implemented by network layer and transport layer. When the host needs to send a packet, it is thrown into the "bucket". The bucket leaks at a constant rate and ensures that receiver receives it at a constant rate. If the bucket is full, the packets cannot be inserted and are lost.

Algorithm

```
1. Start
2. Initialize a counter to n at the tick of the clock.
3. If n is smaller than the packet size of the packet at the head of the queue, go to
   step 7.
4. Pop packet P out of the head of the queue and send it to the network.
5. Decrement the counter by the size of packet P.
6. Go to step 3
7. Reset the counter and go to step 1.
```

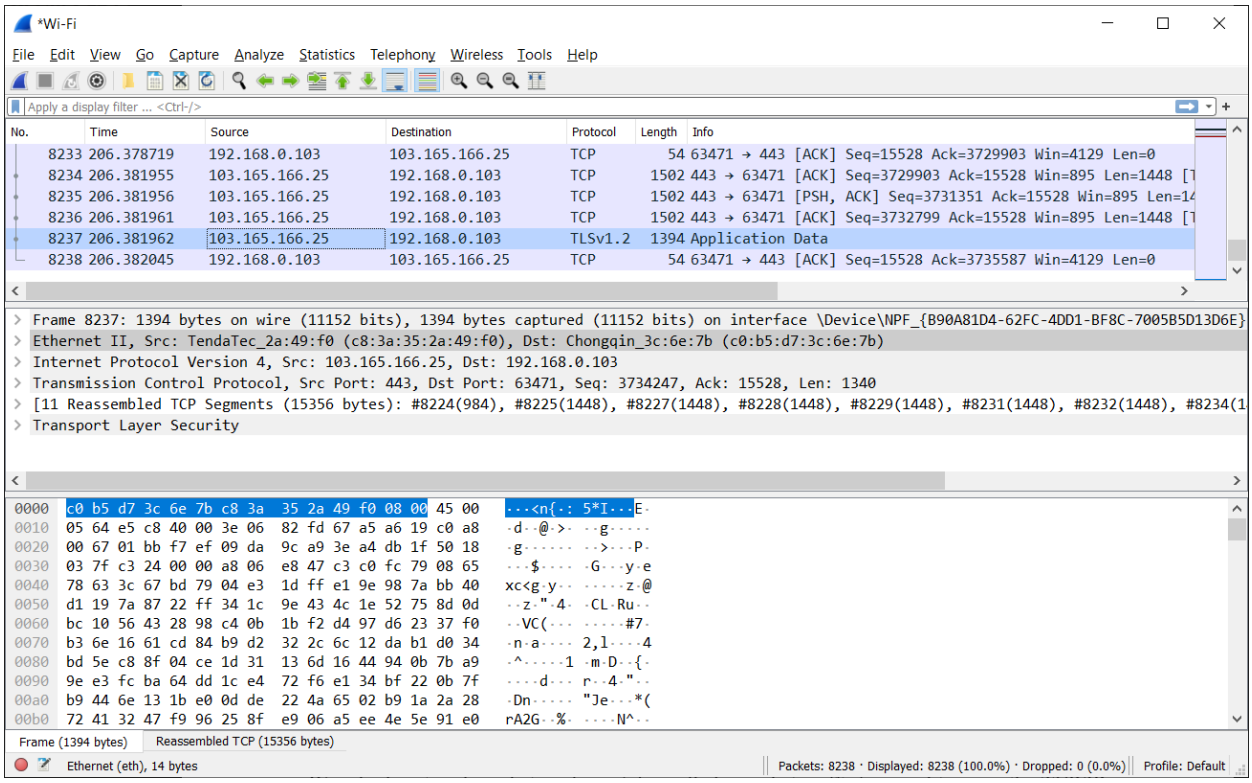
Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define packetCount 10

int main()
{
    srand(time(0));
    int packets[packetCount], i, rate, bucketSize, remainingSize = 0, timeToTransmit, clk,
    op;

    for (i = 0; i < packetCount; ++i)
        packets[i] = (rand() % 6 + 1) * 10;

    printf("\nEnter the Output rate:");
    scanf("%d", &rate);
    printf("Enter the Bucket Size:");
    scanf("%d", &bucketSize);

    i = 0;
    while (i < packetCount || remainingSize > 0)
    {
        if (i < packetCount)
        {
            if ((packets[i] + remainingSize) > bucketSize)
                printf("Bucket capacity exceeded! Packet %d overflow\n", packets[i]);
            else
```

Practice questions

Analyzing data packets on Wireshark

Using Wireshark, capture packets transferred while browsing a selected website



Investigate the protocols used in each packet

Source	Destination	Protocol
192.168.0.103	103.165.166.25	TCP
103.165.166.25	192.168.0.103	TCP
103.165.166.25	192.168.0.103	TCP
103.165.166.25	192.168.0.103	TCP
103.165.166.25	192.168.0.103	TLSv1.2
192.168.0.103	103.165.166.25	TCP

The given packets are transmitted using TCP(Transfer Control Protocol) for transmission. TLS(Transport Layer Security) protocol is used for secure transmission of data over the Internet.

Date: 7.07.22

Experiment 4: Socket programming using UDP

Aim

Implement Client-Server communication using Socket Programming and UDP as transport layer protocol.

Theory

In UDP, the client does not form a connection with the server like in TCP and instead just sends a datagram. Similarly, the server need not accept a connection and just waits for datagrams to arrive. Datagrams upon arrival contain the address of the sender which the server uses to send data to the correct client.

UDP Server :

1. Create a UDP socket.
2. Bind the socket to the server address.
3. Wait until the datagram packet arrives from the client.
4. Process the datagram packet and send a reply to the client.
5. Go back to Step 3.

UDP Client :

1. Create a UDP socket.
2. Send a message to the server.
3. Wait until response from the server is received.
4. Process reply and go back to step 2, if necessary.
5. Close socket descriptor and exit.

Algorithm

Server

```
1. Initiate IP address and port number.

2. Create a UDP socket using socket() system call.

3. If socket() returns negative integer, print error and go to step 7.

4. Bind the local address and port using bind() system call.

5. If bind returns negative integer, print "bind error" and go to step 7.

6. Receive messages from client using recvfrom() system call.

7. Exit.
```

View the active TCP connections in the computer after visiting a website

```
student@CNBIC-AA01-13:~$ netstat -at
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 CNBIC-AA01-13:domain    *:*                     LISTEN
tcp        0      0 localhost:ipp           *:*                     LISTEN
tcp        0    103 192.168.78.213:37988    199.232.253.140:https   FIN_WAIT1
tcp        0    142 192.168.78.213:38274    104.22.25.87:https      FIN_WAIT1
tcp        0    124 192.168.78.213:39164    67.199.150.81:https     FIN_WAIT1
tcp        0    124 192.168.78.213:55590    spcms.pbp.vip.sg3:https FIN_WAIT1
tcp        0      0 192.168.78.213:49120    maa05s22-in-f14.1:https ESTABLISHED
tcp        0      0 192.168.78.213:46370    server-13-227-166:https TIME_WAIT
tcp        0      0 localhost:55902         CNBIC-AA01-13:domain    TIME_WAIT
tcp        0      0 192.168.78.213:52184    34.107.221.82:http      ESTABLISHED
tcp        0      0 192.168.78.213:49184    ec2-34-226-61-176:https TIME_WAIT
tcp        0    103 192.168.78.213:38010    199.232.253.140:https   FIN_WAIT1
tcp        0    103 192.168.78.213:37962    199.232.253.140:https   FIN_WAIT1
tcp        0      0 localhost:55896         CNBIC-AA01-13:domain    TIME_WAIT
tcp        0     35 192.168.78.213:57664    ec2-34-213-33-47.:https ESTABLISHED
tcp        0    124 192.168.78.213:60774    141.226.229.48:https    FIN_WAIT1
tcp        0      0 localhost:55898         CNBIC-AA01-13:domain    TIME_WAIT
tcp        0      0 192.168.78.213:36538    server-13-227-166:https TIME_WAIT
tcp        0    103 192.168.78.213:36858    server-13-227-166:https FIN_WAIT1
tcp        0    103 192.168.78.213:57492    104.244.42.131:https    FIN_WAIT1
tcp        0    103 192.168.78.213:52080    del12s01-in-f3.1e:https FIN_WAIT1
tcp        0      0 192.168.78.213:52258    52.46.136.61:https      TIME_WAIT
tcp        0    124 192.168.78.213:39040    ec2-52-74-68-206.:https FIN_WAIT1
tcp        0      0 192.168.78.213:52260    52.46.136.61:https      TIME_WAIT
tcp        0      0 192.168.78.213:34348    ec2-44-237-168-23:https ESTABLISHED
tcp        0      0 localhost:55900         CNBIC-AA01-13:domain    TIME_WAIT
tcp        0      0 192.168.78.213:52182    34.107.221.82:http      ESTABLISHED
tcp        0      0 192.168.78.213:35916    103.231.98.194:https    TIME_WAIT
tcp        0    272 192.168.99.163:44744    192.168.99.6:ssh        ESTABLISHED
tcp        0      0 192.168.78.213:49146    117.18.237.29:http      ESTABLISHED
tcp       52      0 192.168.99.163:52930    192.168.99.:netbios-ssn ESTABLISHED
tcp        0      0 192.168.78.213:56702    server-65-8-83-14:https TIME_WAIT
tcp        0      0 192.168.78.213:51058    103.229.10.173:https    TIME_WAIT
tcp6       0      0 ip6-localhost:ipp       [::]:*                  LISTEN
```

```
   int port= atoi(argv[1]);
   int sockfd;
   struct sockaddr_in server_addr,client_addr;
   char buffer[1024];
   socklen_t addr_size;
   sockfd = socket(AF_INET, SOCK_DGRAM, 0);
   if (sockfd < 0){
    perror("[-]socket error");
    exit(1);
  }
  memset(&server_addr,'\0',sizeof(server_addr));
  server_addr.sin_family=AF_INET;
  server_addr.sin_port=htons(port);
  server_addr.sin_addr.s_addr=inet_addr(ip);
  int n=bind(sockfd,(struct sockaddr*)&server_addr,sizeof(server_addr));
  if(n<0){
    perror("[-]bind error");
    exit(1);
  }
  while(1){

  bzero(buffer, 1024);
  addr_size = sizeof(client_addr);
  recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr*)&client_addr, &addr_size);
  printf("[+]Data recv: %s\n", buffer);
  bzero(buffer, 1024);

  strcpy(buffer, timeset());

  sendto(sockfd, buffer, 1024, 0, (struct sockaddr*)&client_addr, sizeof(client_addr));
  printf("[+]Data send:Time is  %s\n", buffer);
  }

}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, char **argv){

  if (argc != 2) {
    printf("Usage: %s <port>\n", argv[0]);
    exit(0);
  }

  char *ip = "127.0.0.1";
  int port = atoi(argv[1]);
  int sockfd;
  struct sockaddr_in addr;
  char buffer[1024];
  socklen_t addr_size;

  sockfd = socket(AF_INET, SOCK_DGRAM, 0);
  memset(&addr, '\0', sizeof(addr));
  addr.sin_family = AF_INET;
  addr.sin_port = htons(port);
  addr.sin_addr.s_addr = inet_addr(ip);

  bzero(buffer, 1024);
  strcpy(buffer, "Whats the time server?");
  sendto(sockfd, buffer, 1024, 0, (struct sockaddr*)&addr, sizeof(addr));
  printf("[+]Data send: %s\n", buffer);

  bzero(buffer, 1024);
  addr_size = sizeof(addr);
  recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr*)&addr, &addr_size);
  printf("[+]Data recv: %s\n", buffer);

```

dig

```
→  networklab dig google.com

; <<>> DiG 9.16.1-Ubuntu <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49266
;; flags: qr rd ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             0       IN      A       142.250.196.174

;; Query time: 20 msec
;; SERVER: 192.168.80.1#53(192.168.80.1)
;; WHEN: Wed Apr 27 08:22:06 IST 2022
;; MSG SIZE  rcvd: 54
```

nslookup

```
→  networklab nslookup google.com
Server:         192.168.80.1
Address:        192.168.80.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.196.174
Name:   google.com
Address: 2404:6800:4007:82c::200e
```

route

```
→  networklab route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         DESKTOP-O8J9E17 0.0.0.0         UG    0      0        0 eth0
192.168.80.0    0.0.0.0         255.255.240.0   U     0      0        0 eth0
```

```
    clientCount ++;
  }

  for(int i = 0 ; i < clientCount ; i ++)
    pthread_join(thread[i],NULL);
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <pthread.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>

void *serverCommunication(void *sockID) {
  int clientSocket = *((int *) sockID);

  while(1) {
    char data[1024];
    int read = recv(clientSocket,data,1024,0);
    data[read] = '\0';
    printf("%s\n",data);
  }
}

void main() {
  int clientSocket = socket(PF_INET, SOCK_STREAM, 0);

  struct sockaddr_in server_addr;

  server_addr.sin_family = AF_INET;
  server_addr.sin_port = htons(8080);
  server_addr.sin_addr.s_addr = htonl(INADDR_ANY);

  if(connect(clientSocket,(struct sockaddr*)&server_addr,sizeof(server_addr))==-1)
    exit(0);

  printf("Connection established ............\n");

  pthread_t thread;
  pthread_create(&thread,NULL,serverCommunication,(void*)&clientSocket);

  while(1) {
    char input[1024];
    scanf("%s",input);

    if(strcmp(input,"SEND") == 0) {
      send(clientSocket,input,1024,0);

      scanf("%s",input);
      send(clientSocket,input,1024,0);

      scanf("%[^\n]s",input);
      send(clientSocket,input,1024,0);
    }
  }
}
```

Write a program to show the process id and the parent process id of a process.

Program

```c
// parent child using fork
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char **argv)
{
    int n = fork();
    if (n > 0)
    {
        printf("Parent process with pid:%d\n",n);
    }
    else
    {
        printf("Child process with pid:%d\n",n);
    }
    return 0;
}
```

Output

```
→  practise_questions gcc pid.c -o pid
→  practise_questions ./pid
Parent process with pid:26660
Child process with pid:0
→  practise_questions ./pid
Parent process with pid:26703
Child process with pid:0
→  practise_questions ./pid
Parent process with pid:26708
Child process with pid:0
→  practise_questions
```

Selective Repeat ARQ flow control protocol

Theory

In the selective repeat, the sender sends several frames specified by a window size even without the need to wait for individual acknowledgement from the receiver as in Go-Back-N ARQ. In selective repeat protocol, the re-transmitted frame is received out of sequence. In Selective Repeat ARQ only the lost or error frames are re-transmitted, whereas correct frames are received and buffered. The receiver while keeping track of sequence numbers buffers the frames in memory and sends negative acknowledgement(NACK) for only frames which are missing or damaged. The sender will send/re-transmit a packet for which NACK is received.

Algorithm

Server

```
1. Receive the data packet and buffer the frame in memory.

2. Send the acknowledgement to the sender. Send negative acknowledgement to the missing
or damaged packets.

3. Exit.
```

Client

```
1. Initialise an array of packets.

2. Initialise PACKET_COUNTER to 0.

3. Initialise WINDOW_SIZE.

4. Send the window packets.

5. When acknowledgement is received for the first packet in the window, slide the window.

6. If no acknowledgment is received within the timeout value or negative acknowledgement
is received, send the missing packet again.

7. Repeat the process till all packets are transmitted.

8. Exit.
```

Program

Server

```c
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define packetCount 10

typedef struct Packet
{
    int id;
    int received;
} Packet;

Packet packets[packetCount];
```

Output

```
client  client.c  server  server.c
s1923@administrator-rusa:~/network/chat-using-thread$ gcc server.c -p
thread -o server
s1923@administrator-rusa:~/network/chat-using-thread$ ./server 127.0.
0.1
[+]TCP server socket created.
[+]Bind to the port number: 5566
Listening...
^C
s1923@administrator-rusa:~/network/chat-using-thread$ gcc server.c -p
thread -o server
s1923@administrator-rusa:~/network/chat-using-thread$ ./server 127.0.
0.1
Server started listenting on port 8080 ...........
Client 1 connected.
Client 2 connected.
Client 3 connected.
```

```
s1923@192.168.99.6's password:

   !!!!! Welcome to Linux Server !!!!!

Last login: Wed May 18 14:30:18 2022 from 192.168.99.208
s1923@administrator-rusa:~$ cd network/chat-using-thread
s1923@administrator-rusa:~/network/chat-using-thread$ ls
client  client.c  server  server.c
s1923@administrator-rusa:~/network/chat-using-thread$ gcc client.c -pthre
o client
s1923@administrator-rusa:~/network/chat-using-thread$ ./client
Connection established ............
SEND 2 HELLO
 HAI
 THIS IS CLIENT 3
SEND 3 HELLO CLIENT 3
```

```
s1923@administrator-rusa: ~/network/chat-using-thread
File  Edit  View  Search  Terminal  Help

   !!!!! Welcome to Linux Server !!!!!

Last login: Wed May 18 14:47:28 2022 from 192.168.99.208
s1923@administrator-rusa:~$ cd network/chat-using-thread/
s1923@administrator-rusa:~/network/chat-using-thread$ ls
client  client.c  server  server.c
s1923@administrator-rusa:~/network/chat-using-thread$ ./client.c
-bash: ./client.c: Permission denied
s1923@administrator-rusa:~/network/chat-using-thread$ ./client
Connection established ............
SEND 1 THIS IS CLIENT 3
SEND 2 THIS IS CLIENT 3
 HELLO CLIENT 3
 HELLO CLIENT 3
```

```
s1923@administrator-rusa: ~/network/chat-using-thread
File  Edit  View  Search  Terminal  Help
Segmentation fault (core dumped)
s1923@administrator-rusa:~/network/chat-using-fork$ cd .
s1923@administrator-rusa:~/network/chat-using-fork$ cd ..
s1923@administrator-rusa:~/network$ ls
a.out            chat-using-thread  client.c  server     server.c
chat-using-fork  client             fork      server1.c
s1923@administrator-rusa:~/network$ cd chat-using-thread/
s1923@administrator-rusa:~/network/chat-using-thread$ ls
client  client.c  server  server.c
s1923@administrator-rusa:~/network/chat-using-thread$ ./client
Connection established ............
 HELLO
SEND 1 HAI
 THIS IS CLIENT 3
SEND 3 HELLO CLIENT 3
```

10. Receive message sent by client using recv() system call.

11. Print the received message.

12. Send reply using send() system call.

13. Close the connection using close() system call and go to step 8.

14. Exit.

Client

1. Initialise IP address and port number.

2. Create a socket using socket() system call.

3. Print "TCP socket created".

4. Connect to the listening socket using connect() system call.

5. Print "Connected to the server".

6. Send a message to the server using send() system call.

7. Receive reply using recv() system call.

8. Print the received message.

9. Close the connection.

10. Print "Disconnected from server".

11. Exit

Program

Server

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

void main(){
  char *ip = "127.0.0.1";
  int port = 5566;

  int server_sock, client_sock;
  struct sockaddr_in server_addr, client_addr;
  socklen_t addr_size;
  char buffer[1024];
  int n;

  server_sock = socket(AF_INET, SOCK_STREAM, 0);
  if (server_sock < 0){
    perror("[-]Socket error");
    exit(1);
  }
  printf("[+]TCP server socket created.\n");

  memset(&server_addr, '\0', sizeof(server_addr));
  server_addr.sin_family = AF_INET;
```

iftop

```
                    12.5Kb              25.0Kb              37.5Kb              50.0Kb              62.5Kb
pop-os                                  => whatsapp-cdn-shv-01-maa2.fbcdn.net          1.59Kb   651b   465b
                                        <=                                             2.22Kb   936b   669b
pop-os                                  => _gateway                                     284b   279b   322b
                                        <=                                              964b   539b   449b
pop-os                                  => server-108-159-15-2.maa51.r.cloudfront.net   208b   166b   149b
                                        <=                                              448b   358b   320b
224.0.0.251                             => 192.168.0.2                                    0b     0b     0b
                                        <=                                                0b   439b   314b
pop-os                                  => server-108-159-15-63.maa51.r.cloudfront.net    0b   125b   119b
                                        <=                                                0b   269b   256b
pop-os                                  => server-108-159-15-102.maa51.r.cloudfront.net 208b   125b   119b
                                        <=                                              448b   269b   256b
pop-os                                  => 64.52.120.34.bc.googleusercontent.com        332b   133b    95b
                                        <=                                              524b   210b   150b
255.255.255.255                         => 192.168.0.5                                    0b     0b     0b
                                        <=                                                0b   144b   103b
pop-os                                  => maa05s21-in-f10.1e100.net                      0b    49b   325b
                                        <=                                                0b    86b   839b
pop-os                                  => maa03s47-in-f1.1e100.net                     208b    42b    30b
                                        <=                                              208b    42b    30b
all-systems.mcast.net                   => _gateway                                       0b     0b     0b
                                        <=                                                0b    26b    18b
pop-os                                  => 224.0.0.251                                    0b    26b    18b
                                        <=                                                0b     0b     0b
pop-os                                  => maa05s20-in-f10.1e100.net                      0b     0b   286b
                                        <=                                                0b     0b   226b
```

whois

```
aswin@DESKTOP-O8J9E17:/mnt/c/Users/Aswin Jayaji/networklab$ whois DESKTOP-O8J9E17

#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2022, American Registry for Internet Numbers, Ltd.
#


#
# Query terms are ambiguous.  The query is assumed to be:
#     "e / desktop-o8j9e17"
#
# Use "?" to get help.
#

No match found for desktop-o8j9e17.


#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2022, American Registry for Internet Numbers, Ltd.
#
```

mtr

```
                                   My traceroute  [v0.93]
DESKTOP-O8J9E17 (127.0.0.1)                                                    2022-04-27T12:46:23+0530
Keys:  Help   Display mode   Restart statistics   Order of fields   quit

                              Last 158 pings
 1. localhost                                                                  .>........................

Scale:  .:0 ms  1:0 ms  2:0 ms  3:0 ms  a:0 ms  b:0 ms  c:0 ms  >
```

Contents

Cycle 1 **4**
1.1. Familiarization of Linux Commands . . . 4
1.2. Familiarization of System Calls . . . 18

Cycle 2 **27**
2.1. Socket programming using TCP . . . 27
2.1.1. Multi-User Chat Server . . . 31
2.1.2. Web Proxy Server . . . 35
Send the reverse of client message as server response. . . . 41
Convert the client message to uppercase and send as server response. . . . 43
Practice Questions . . . 46
Concurrent Time Server using UDP . . . 46
2.3. SMTP . . . 49
2.4. FTP . . . 53

Cycle 3 **58**
3.1. Sliding Window Protocols . . . 58
3.1.1. Simple Stop-and-Wait flow control protocol . . . 58
3.1.2. Stop-and-Wait ARQ flow control protocol . . . 61
3.1.3. Go-Back-N ARQ flow control protocol . . . 64
3.1.4. Selective Repeat ARQ flow control protocol . . . 68

Cycle 4 **73**
4.1. Distance Vector Routing Protocol . . . 73
4.2. Link State Routing Protocol . . . 75

Cycle 5 **78**
5.1. Leaky Bucket . . . 78

Cycle 6 **80**
6.1. Wireshark . . . 80

Cycle 7 **87**
7.1. Network Services . . . 87

Cycle 8 **96**
8.1. NS2 . . . 96

```c
    time(&t);
    sprintf(header, "Date: %s", ctime(&t));
    sprintf(header + strlen(header), "From: %s\n", from);
    sprintf(header + strlen(header), "To: %s\n", to);
    sprintf(header + strlen(header), "----------------------------------------\n\n");
    sprintf(header + strlen(header), "%s", content);

    return header;
}

void main()
{
    int server_sock;
    struct sockaddr_in server_addr;
    socklen_t addr_size;
    char buffer[2000];

    char subject[100], sender[100], recepient[100], content[1000];

    server_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (server_sock < 0)
    {
        perror("Socket error\n");
        exit(1);
    }

    memset(buffer, '\0', 2000);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(25);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.100");

    connect(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr));
    printf("Connected to server\n");

    printf("<-- Send a mail -->\n");
    strcpy(sender, "client@");
    strcat(sender, domain);
    printf("Enter recepient mail: ");
    scanf("%[^\n]%*c", recepient);
    printf("Enter subject: ");
    scanf("%[^\n]%*c", subject);
    printf("Enter content: ");
    scanf("%[^\n]%*c", content);

    char *mail = mailHeader(sender, recepient, subject, content);
    printf("\nMail contents:\n");
    printf("**********************************************\n");
    printf("%s\n", mail);
    printf("**********************************************\n");

    bzero(buffer, 2000);
    strcpy(buffer, mail);
    send(server_sock, buffer, 2000, 0);
    printf("Mail sent\n");

    close(server_sock);
}
```

Write the system calls used for creating sockets and transferring data between two nodes.

```
Socket Programming system calls in c

1. Socket Creation
Eg: int sockfd = socket(domain, type, protocol)

sockfd: socket descriptor, an integer (like a file-handle)
domain: Integer that specifies communication domain

type: communication type,
        Eg:-
        SOCK_STREAM: TCP(reliable, connection oriented)
        SOCK_DGRAM: UDP(unreliable, connectionless)

protocol: Protocol value for Internet Protocol(IP), which is 0.


2. Binding
Eg: bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)

After creation of the socket, bind function binds the socket to the address and port number
specified in addr


3. Accept
Eg: new_socket= accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)

It extracts the first connection request on the queue of pending connections for the listening
socket, sockfd, creates a new connected socket, and returns a new file descriptor referring to
that socket. This establishes a connection between two nodes.


4. Connection
Eg: connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)

The connect() system call connects the socket referred to by the file descriptor sockfd to the
address specified by addr. Server address and port is specified in addr.
```

Result

The system calls were executed and output was verified.

```
    struct sockaddr_in addr;
    char buffer[1024];
    socklen_t addr_size;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0)
    {
        perror("[-]Socket error");
        exit(1);
    }

    memset(&addr, '\0', sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    addr.sin_addr.s_addr = inet_addr(ip);

    int packets[packetCount];
    for (int i = 0; i < packetCount; ++i)
        packets[i] = i + 1;

    int windowStart = 0;
    int windowEnd = windowStart + windowSize;

    int flag = 1;
    sendWindowPackets(&sockfd, addr, packets, windowStart, windowEnd);
    while (windowStart != windowEnd)
    {
        bzero(buffer, 1024);
        addr_size = sizeof(addr);

        struct timeval timeout;
        timeout.tv_sec = timeoutValue;
        timeout.tv_usec = 0;

        if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeout, sizeof(timeout)) < 0)
            printf("Error\n");

        else
        {
            bzero(buffer, 1024);
            addr_size = sizeof(addr);

            int rec = recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr *)&addr, &addr_size);
            if (rec < 0)
            {
                printf("Client: Timeout error! Sending window packets again\n");
                sendWindowPackets(&sockfd, addr, packets, windowStart, windowEnd);
            }
            else
            {
                printf("Client: Received acknowledgement for packet %s\n", buffer);
                if(atoi(buffer)!=packets[windowStart]) {
                  printf("Client: Wrong acknowledgement! Sending window packets again\n");
                  sendWindowPackets(&sockfd, addr, packets, windowStart, windowEnd);
                }
                else {
                  windowStart++;
                  if (windowEnd < packetCount)
                      windowEnd++;

                  if (windowStart != windowEnd)
                  {
                      bzero(buffer, 1024);
                      if(flag==1) {
                        sprintf(buffer, "%d", packets[windowEnd-1]);
                        printf("Client: Sending packet %s\n", buffer);
                        sendto(sockfd, buffer, 1024, 0, (struct sockaddr *)&addr, sizeof(addr));
                      }
                      if(windowEnd=packetCount)
                        flag=0;
                  }
              }
            }
```

Experiment no: 2.4
Date: 10.06.22

FTP

Aim

Develop a concurrent file server which will provide the file requested by a client if it exists. If not, the server sends appropriate messages to the client. Server should also send its process ID (PID) to clients for display along with the file or the message.

Theory

FTP stands for File transfer protocol.It is a standard internet protocol provided by TCP/IP used for transmitting the files from one host to another. It is mainly used for transferring the web page files from their creator to the computer that acts as a server for other computers on the internet. It is also used for downloading the files to computer from other servers.

Algorithm

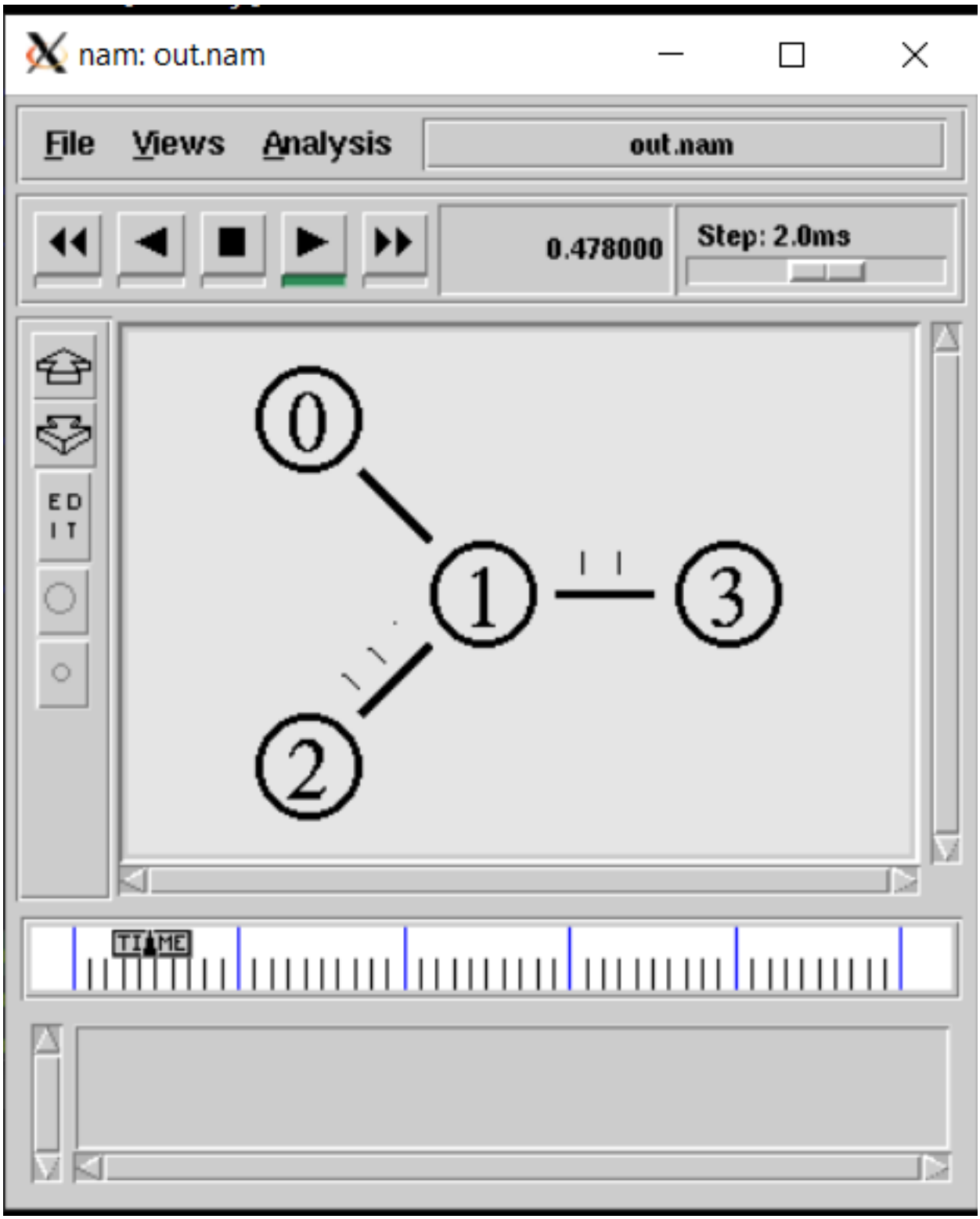
Server

```
1. Initialise IP address and port number.

2. Create a TCP socket using socket() system call.

3. Bind the socket to local address using bind() system call.

4. Listen to client using listen() system call.

5. Accept a connection from client using accept() system call.

6. Receive the filename from client using recv() system call.

7. If the requested file is in the server, send it to client along with the process id
using send() system call. Else sent a message "File is not found".

7. Exit.
```

Client

```
1. Initialise IP address and port number.

2. Create a socket using socket() system call.

3. Connect to server using connect() system call.

4. Request a file to server using send() system call.

5. Receive the process id along with either the file or error message from the server
using recv() system call.

6. Exit
```

Output



Result

The usage of the NS2 utility was studied.

```
$ftp attach-agent $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n1 $sink

$ns connect $tcp $sink

$ns rtmodel-at 1.0 down $n0 $n1
$ns rtmodel-at 2.0 up $n0 $n1
$ns rtproto DV
$ns at 0.0 "$ftp start"
$ns at 150.0 "finish"
$ns run
```

AWK program

```
BEGIN{
    pkt=0;
    time=0;
}
{
    if ($1=="r" && $3=="0" && $4=="1")
    {
        pkt=pkt+$6;
        time=$2;
        printf("%f\t %d\n", time, pkt);
    }
}
END {
    printf("throughput fMbps", ((pkt/time) *(8/1024)));
}
```

2. Convert the client message to uppercase and send as server response.

Server

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <ctype.h>


#define MAXLINE 1024

void to_upper(char *s)
{

    int i;
    for (i = 0; s[i]!='\0'; i++) {
      if(s[i] >= 'a' && s[i] <= 'z') {
         s[i] = s[i] -32;
      }
   }
}

int main() {
  char *ip="127.0.0.8";
  int port=5513;

  int sockfd;
  char buffer[MAXLINE];
  //char rev[MAXLINE];
  char *hello = "Hello from server";
  struct sockaddr_in servaddr, cliaddr;

  sockfd=socket(AF_INET,SOCK_DGRAM,0);

  if ( sockfd < 0 ) {
    perror("[-]Socket error");
    exit(0);
  }
  printf("[+]UDP server socket created");

  memset(&servaddr, 0, sizeof(servaddr));
  memset(&cliaddr, 0, sizeof(cliaddr));


  servaddr.sin_family = AF_INET;
  servaddr.sin_addr.s_addr = inet_addr(ip);
  servaddr.sin_port = port;


  if ( bind(sockfd, (const struct sockaddr *)&servaddr,sizeof(servaddr)) < 0 )
  {
    perror("[-]Bind error");
    exit(1);
  }

  int len, n;

  len = sizeof(cliaddr);

  n = recvfrom(sockfd, (char *)buffer, MAXLINE,MSG_WAITALL, ( struct sockaddr *) &cliaddr,&
    len);
  buffer[n] = '\0';
  printf("Client : %s\n", buffer);

  to_upper(buffer);
```

Practice questions

View the configuration, including addresses of your computers' network interfaces

```
s1923@administrator-rusa:~$ ifconfig
eno1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.99.6  netmask 255.255.255.0  broadcast 192.168.99.255
        inet6 fe80::a94:efff:fe5c:d998  prefixlen 64  scopeid 0x20<link>
        ether 08:94:ef:5c:d9:98  txqueuelen 1000  (Ethernet)
        RX packets 17791479  bytes 1421569934 (1.4 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 82247847  bytes 120513842671 (120.5 GB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 16

eno2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.78.250  netmask 255.255.255.0  broadcast 192.168.78.255
        inet6 fe80::6411:8b69:de8:11a2  prefixlen 64  scopeid 0x20<link>
        ether 08:94:ef:5c:d9:99  txqueuelen 1000  (Ethernet)
        RX packets 183517  bytes 108943285 (108.9 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 60658  bytes 8159957 (8.1 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 17

eno3: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 08:94:ef:5c:d9:9a  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 16

eno4: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 08:94:ef:5c:d9:9b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 17

enp0s20u13u5: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        ether 0a:94:ef:5c:d9:9f  txqueuelen 1000  (Ethernet)
        RX packets 3906  bytes 110250 (110.2 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 4407137  bytes 114810673029 (114.8 GB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4407137  bytes 114810673029 (114.8 GB)
```

Write a c program to implement file manipulation system calls.

Program

```c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>


int main (int argc, char **argv){
    FILE *file = fopen("file.txt", "r+");
    char line[100]; // stack smashed on overflow
    // memset(line, 1, 100);

    if (file == NULL){
        printf("Error opening file\n");
        return 1;
    }
    printf("%s\n",line);
    while(fscanf(file, " %[^\n]", line)!=EOF){
        printf("%s\n",line);
    }
    fclose(file);
    return 0;
}
```

Output

```
→  practise_questions gcc fileman.c -o fileman
→  practise_questions ./fileman

Most of the time, most variables in most threads are local, meaning that they
belong to a single thread and no other threads can access them. As long as
that's true, there tend to be few synchronization problems, because threads
just don't interact.
But usually some variables are shared among two or more threads; this
is one of the ways threads interact with each other. For example, one way
to communicate information between threads is for one thread to read a value
written by another thread.
If the threads are unsynchronized, then we cannot tell by looking at the
program whether the reader will see the value the writer writes or an old value
that was already there. Thus many applications enforce the constraint that
the reader should not read until after the writer writes. This is exactly the
serialization problem in Section 1.3.
Other ways that threads interact are concurrent writes (two or more writers)
and concurrent updates (two or more threads performing a read followed
by a write). The next two sections deal with these interactions. The other
possible use of a shared variable, concurrent reads, does not generally create a
synchronization problem.
```

Practice Questions

Implement a multi-user chat server using TCP as transport layer protocol.

Algorithm

Server

```
1. Initialise IP address and port number.

2. Create a TCP socket using socket() system call.

3. If socket() returns negative integer, print error and go to step 13.

4. Print "Server socket created!".

5. Bind the local address and port using bind() system call.

6. If bind returns negative integer, print "bind error" and go to step 13.

7. Listen to client using listen() system call.

8. Accept connection from client using accept() system call.

9. Print "Client connected".

10. Receive message sent by client using recv() system call.

11. Pass the message to the intended receiver using send() system call.

12. Close the connection using close() system call and go to step 8.

13. Exit.
```

Client

```
1. Initilise IP address and port number.

2. Create a socket using socket() system call.

3. Print "TCP socket created".

4. Connect to the listening socket using connect() system call.

5. Print "Connected to the server".

6. Send a message along with the intended receiver to the server using send() system call.

7. Receive reply using recv() system call.

8. Print the received message.

9. Close the connection.

10. Print "Disconnected from server".

11. Exit
```

Program

Server

```
    while (1)
    {
        printf("\n1. Upload a file(PUT)\n2. Download a file(GET)\n3. Exit(BYE)\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            bzero(buffer, 1024);
            strcpy(buffer, "PUT");
            printf("Enter filename: ");
            scanf("%s", filename);
            fptr = fopen(filename, "r");
            if (!fptr)
                perror("File doesn't exist!\n");
            else
            {
                send(server_sock, buffer, 1024, 0);
                bzero(buffer, 1024);
                strcpy(buffer, filename);
                send(server_sock, buffer, 1024, 0);

                bzero(buffer, 1024);
                while (!feof(fptr))
                {
                    bzero(buffer, 1024);
                    fscanf(fptr, "%[^\n] ", buffer);
                    send(server_sock, buffer, 1024, 0);
                }
                bzero(buffer, 1024);
                strcpy(buffer, "END$");
                send(server_sock, buffer, 1024, 0);
                fclose(fptr);
                printf("File %s sent\n", filename);
            }
            break;

        case 2:
            bzero(buffer, 1024);
            strcpy(buffer, "GET");
            send(server_sock, buffer, 1024, 0);
            bzero(buffer, 1024);
            recv(server_sock, buffer, 1024, 0);
            printf("Process id: %s\n", buffer);

            printf("Enter filename: ");
            scanf("%s", filename);
            bzero(buffer, 1024);
            strcpy(buffer, filename);
            send(server_sock, buffer, 1024, 0);

            bzero(buffer, 1024);
            recv(server_sock, buffer, 1024, 0);

            if (strcmp(buffer, "404") == 0)
                printf("File does not exist in remote server\n");
            else if (strcmp(buffer, "200") == 0)
            {
                fptr = fopen(filename, "w");
                while (1)
                {
                    recv(server_sock, buffer, 1024, 0);
                    if (strcmp(buffer, "END$") == 0)
                        break;
                    fprintf(fptr, "%s", buffer);
                    bzero(buffer, 1024);
                }
                fclose(fptr);
                printf("File %s received\n", filename);
            }
            break;
        case 3:
            bzero(buffer, 1024);
            strcpy(buffer, "BYE");
```